

UNIVERSIDAD
DE GRANADA

Práctica 3

Problema del portfolio

Pablo Linari Pérez

DNI: 75571079W
Grupo de prácticas: 3 (Martes 15:30-17:30)
Curso: 25/26
Correo: pablolinari@correo.ugr.es

Índice de contenidos

1 Descripción del problema 3
2 Algoritmos empleados y estructura del problema 4
2.1 Estructura del problema 4
2.2 Representación de las soluciones 4
2.3 Funciones comunes usadas por los algoritmos 5
2.4 Operadores 7
3 Estructura de los algoritmos 8
3.1 Local Search 8
3.2 Algoritmo BMB 9
3.3 Algoritmo ILS 10
3.4 Algoritmo Enfriamiento simulado 11
3.5 Algoritmo ILS-ES 13
4 Estructura del código 13
4.1 Organización de archivos 13
4.2 Compilación 14
4.3 Manual de usuario: Ejecución 15
5 Experimentos y análisis de los resultados 16
5.1 Casos empleados y valores de los parámetros. 16
5.2 Resultados obtenidos 16
5.3 Boxplots de los resultados 17
5.4 Gráficas de evolución del fitness durante las ejecuciones 19
6 Análisis de los resultados 21
6.1 Comparación global de fitness (2015-2024) y gráficas 21
6.1.1 Enfriamiento simulado 21
6.1.2 ILS vs BMB vs ILS-ES 21
6.1.3 Comparación general 22
6.2 Resultados en 2025 y beneficio 22
6.3 Conclusiones 22

1 Descripción del problema

El problema que se ha escogido es el problema del portfolio, en el cual se nos da una lista de empresas de las cuales sabemos su retorno obtenido diario desde principios de 2015 hasta finales de 2025. Con estos datos necesitamos elaborar una solución indicando qué porcentaje de nuestra cartera deberíamos haber invertido en cada empresa para maximizar el beneficio de las inversiones.

En este contexto, el problema no solo busca el máximo retorno, sino que también suele considerar la correlación entre activos, teniendo esto en cuenta en las ecuaciones que se usan para maximizar el beneficio.

De esta manera podemos ajustar el programa ajustando el parámetro λ de la ecuación a maximizar para así tener mayor o menor aversión al riesgo.

2 Algoritmos empleados y estructura del problema

2.1 Estructura del problema

Para modelar el problema del portfolio se ha usado la clase Pportfolio la cual es hija de la clase Problem que viene ya predefinida con varios métodos virtuales, los cuales se implementan en la clase hija. Para representar el problema usando estas clases se tienen en cuenta los siguientes atributos:

- **days** : número entero de días de los que se tienen datos.
- **ncompany** : número entero de empresas de las que se tienen datos.
- **low** : mínimo porcentaje de inversión en una empresa (double).
- **high** : máximo porcentaje de inversión en una empresa (double).
- **lambda** : parámetro entero usado en el fitness.
- **data** : matriz de la librería Eigen con valores double que contienen los beneficios diarios de cada empresa, en las columnas se representan las empresas y en las filas los días.
- **covarianceMat** : matriz de la librería Eigen con valores double que contiene la covarianza de las empresas.
- **logSums** : vector de la librería Eigen con valores double que contiene la suma por columnas después de aplicar **log(valor+1)** a cada posición de la matriz.
- **ERROR** : Error permitido en la suma total del vector solución (double).

Podemos observar que se ha optado por usar la librería Eigen de C++ para poder manejar las operaciones de matrices y vectores como la suma total, la media total, la media por columnas o por filas, multiplicación escalar a escalar, etc. Esto hace que el código sea más compacto y fácil de seguir a la vez que más preciso y cómodo de utilizar.

También se han guardado los datos como **covarianceMat** y **logSums** que son constantes durante la ejecución del programa y solo cambian cuando cambia el archivo de entrada. Al ser valores calculados con operaciones matriciales son costosos de calcular y eso hace que el programa sea más rápido.

2.2 Representación de las soluciones

Para este problema las soluciones se representan por vectores de la biblioteca Eigen con tipo de dato double donde cada posición del vector representa la i-ésima empresa del fichero de datos de entrada, todos estos vectores deben cumplir las siguientes restricciones:

- El tamaño de la solución debe ser el mismo que el número de empresas en los datos de entrada.
- La suma de todos sus valores debe ser igual a 1.
- Cada posición del vector tiene que tener un valor entre **low** y **high**
- Puede haber posiciones del vector con valor 0 (representando que en esa empresa no se invierte).

La mejor solución será aquella que consiga maximizar la función objetivo.

2.3 Funciones comunes usadas por los algoritmos

En primer lugar la función que permite cargar los datos en nuestra clase Pportfolio es una función implementada en un archivo auxiliar ya que no tiene por qué pertenecer necesariamente al problema que se plantea. Esta función lee datos de un fichero en formato CSV. Tras abrir el archivo la función descarta la primera línea con los nombres de las empresas y comienza a leer las siguientes líneas, una vez escogida una línea, se salta el string que contiene la fecha y comienza a insertar datos al vector usando como separador la coma. El código comprueba los valores de la fecha leída, si el parámetro only2025 es True, solo se leen datos de 2025, de lo contrario se leen todos los datos hasta el último día del 2024. Una vez leídos todos los datos, la matriz obtenida se convierte a una matriz Eigen para poder manejarla con más facilidad.

```
funcion LoadCSV(archivo, only2025):
  matrix data
  leer(primera_linea)
  abrir archivo
  while(leer(linea_archivo)):
    if es fecha el primer string :
      continue
    if(es fecha el primer string and not only2025):
      solo leer valores de 2014-2024
    if(es fecha el primer string and only2025):
      solo leer valores de 2025
    vector valores

     while(leer(dato)):
       if(dato no vacío):
         valores.pushback(dato)
       else:
         valores.pushback(0.0)

    data.pushback(valores)

  cerrar archivo

   if(data no vacío):
     matriz_eigen = data
  return matriz_eigen
```

Las principales funciones que se usan de la clase **Pportfolio** para implementar los algoritmos son las siguientes:

La función **createSolution** se usa en dos algoritmos, en la Búsqueda Aleatoria para generar soluciones aleatorias y escoger finalmente la que mejor fitness tenga y en la Búsqueda Local para generar la solución desde la cual se comienza a buscar la mejor solución posible.

```
funcion createSolution():
   vector solución;

   for i<solución.size :
```

```
      solución(i) = Aleatorio entre high y low


    numeroceros = Aleatorio entre 1 y solución.size-1
    vectorindices = {0,1,2, ......,solución.size-1}
   desordenar(vectorindices)
    ## Seleccionamos valores aleatorios que ponemos a 0
    for i<numeroceros:
      solución(vectorindices[i])=0

    solución = solución/solución.sum
    if la solución no es válida
      fix(solución)

    return solución;
```

```
# Pseudocódigo del arreglo de solución
 funcion fix(solución):
    solución = solución/solución.sum

 for i in ncompany:
   if solución(i) < low:
     solución(i) = 0
   else if solución(i) > high:
     solución(i) = this->high


 double diff = 1.0 - solución.sum
 while diff >ERROR and no supera máximo de iteraciones:
  ajustado = false

   while i < ncompany and not ajustado:
     if solución(i) en rango [low, high):
       espacio = high - solución(i)
      if hay espacio :
        sumar diferencia y ajustado=true
       else :
         solución(i) = high
        diff =diff- espacio

   if no se ha ajustado y diferencia menor que low:
     for i in ncompany and not ajustado:
       if solución(i) == 0:
         solución(i) = mínimo entre diff y high
        ajustado = true

   if no se ha ajustado y la diferencia es menor que low:
     while i < ncompany and not ajustado:
       if (solución(j) > low and solución(j) + diff < high:
         solución(j) += diff
        ajustado = true

   diff = 1.0 - solución.sum
```

La solución fix se aplica cada vez que se genera una solución aleatoria para arreglarla, lo que se hace es tras normalizar la solución modificar los valores fuera de rango para que cumplan las condiciones de ser solución. Una vez hecho esto, si hace falta añadir más valor en alguna posición se añade, primero se comprueba si se puede repartir dentro de pesos que ya estén activos, de no ser así si la diferencia es mayor que el límite inferior se activa un cero de la solución. En caso de que la diferencia sea menor que el mínimo permitido se intenta distribuir la diferencia entre valores que ya estén activos y tengan un margen para ser de nuevo ajustados. Este proceso se repite hasta que la diferencia es menor que el error permitido o se sobrepasen las evaluaciones. En el código las evaluaciones permitidas son altas para que no haya problema por la precisión de las operaciones.

```
# Pseudocódigo del cálculo del beneficio
 funcion profit(solution):
   return (solution * logSums).sum #multiplicación componente a componente
```

```
# Pseudocódigo del cálculo del riesgo
 funcion risk(solution):
   return (solution.transpose * covarianceMat * solution);
#devuelve un número resultado de las dimensiones 1xn * nxn * nx1
```

```
# Pseudocódigo del cálculo del fitness
 funcion fitness(solution):
   return (profit - lambda*risk);
#devuelve un número resultado de las dimensiones 1xn * nxn * nx1
```

El principal objetivo de los algoritmos es maximizar la función fitness, la cual trata de maximizar el beneficio y minimizar el riesgo en una sola fórmula. El parámetro lambda se usará a modo de ajuste, cuando más alto sea su valor el algoritmo valorará mejor soluciones con poco beneficio pero que sean seguras, es decir, que tengan poco riesgo, mientras que cuanto más cercano a 0 sea lambda, el algoritmo valorará mejor las soluciones con máximo beneficio posible, ignorando el riesgo.El valor que se usa durante la práctica es $\lambda = 500$ que es el indicado en el guión.

2.4 Operadores

En esta práctica el único operador que se utiliza es el operador de mutación el cual vemos a continuación.

```
# Pseudocódigo del operador de mutación
funcion mutation(mutSol, ratio):
  solucion = mutSol
  nselection = ceil(ratio * ncompany)
  index = aleatorio entre 0 y ncompany-1

  if index + nselection > ncompany:
    endindex = index
    index = endindex - nselection
  else:
    endindex = index + nselection
```

```
  desordenar(solucion[index:endindex])
  return solucion
```

El operador de mutación se utiliza en el ILS y consiste en desordenar el 20% de las posiciones de un vector solución seleccionadas en un rango aleatorio.

3 Estructura de los algoritmos

3.1 Local Search

Es el mismo que la práctica anterior , pero al aplicarlo modificaremos el ratio y la solución inicial que toma.

El algoritmo de búsqueda local (el primero el mejor) parte de una solución generada aleatoriamente y genera un vecindario alrededor de esta solución, una vez generado el vecindario, se comienza a recorrer cada vecino y se evalúa su fitness. Si se encuentra un fitness mejor que el de la solución actual, se escoge ese vecino como mejor solución y se genera un nuevo vecindario alrededor de él. El proceso termina cuando se llega al máximo de evaluaciones permitidas o cuando no se ha encontrado ningún vecino del vecindario actual generado que mejore la solución.

```
# Pseudocódigo del algoritmo de búsqueda local
función LocalSearch(max_evals):
    double ratio
     solucioninicial = solucionAleatoria
     bestFitness = fitness(solucionAleatoria)

    vecinos = Generar vector de todas las parejas (i,j) posibles con i,j en
[0,ncompany-1]
    # excepto en el caso i=j;

    while evals < maxevals and haymejora:

      mezclar vecinos
      for v in vecinos:
        cantidadmover = ratio*solucioninicial[v.primero]

        solucioninicial[v.primero]-=cantidadmover
        solucioninicial[v.segundo]+=cantidadmover

         if es válida solucioninicial:
          evals = evals+1

           if fitness(solucioninicial) > mejorfitness:
             haymejora = true
             mejorfitness = fitness de la solución inicial modificada
             #si hay una mejora salimos del bucle for
             #ya que es una búsqueda del primero el mejor
            salir del bucle for
```

```
        # si no cambiamos de solución inicial, hay que
        # devolverla a su estado original
        devolver solucioninicial a su estado original
        if evals==maxevals:
         salir del bucle for
         #para no pasarnos de evaluaciones
    return solucioninicial, mejorfitness, evals
```

```
   # Generación del vecindario
  for i,j in ncompany:
    vectorparejas.insertar(pareja(i,j))
```

La implementación del algoritmo de Búsqueda local se divide en dos partes, primero se crea el vector vecinos, en este caso se usa un vector de parejas, donde cada vecino es la pareja (i,j) la cual nos indicará los índices que debemos modificar en el vector solución. Una vez creado el vector, lo mezclamos aleatoriamente, y este será el que nos indique qué vecinos debemos de probar. Si no fuese una búsqueda del primero el mejor y se escogiese el mejor vecino no haría falta mezclar las parejas, ya que el orden no importaría porque siempre tendríamos que recorrer todo el vecindario.

Una vez disponemos del vecindario, debemos generar las nuevas soluciones a partir de las parejas que vamos escogiendo del vector. Si tenemos la pareja (i,j) quitaremos un porcentaje del peso a la posición i y se la sumaremos a la posición j (el ratio usado en la práctica es 0.4). Esto nos genera una nueva solución la cual comprobaremos si es válida y, en caso de ser válida, si supera el fitness de nuestra mejor solución hasta el momento. Este nuevo vector solución con las nuevas modificaciones en los índices (i,j) se convertirá en nuestra nueva solución. Dejaremos de recorrer el vector de vecinos y volveremos a generar un vecindario mezclado aleatoriamente para recorrerlo de nuevo desde el principio para ver si a partir de esta nueva solución generada conseguimos una mejor.

Si la solución generada no es válida o no mejora la solución actual, restauraremos las modificaciones hechas al vector solución. De esta manera podemos usar un solo vector y modificar dos posiciones en vez de hacer copias de vectores completos.

Mientras que no se superen las restricciones de las evaluaciones y se sigan encontrando vecinos mejores en cada vecindario seguiremos intentando encontrar la mejor solución. En caso de recorrer todo el vecindario y no encontrar una solución mejor o que se superen las evaluaciones permitidas, se detendrá la búsqueda.

3.2 Algoritmo BMB

```
function bmb(maxevals):
  bestsolution = none
  bestfitness = -infinito
  numsol = numero de soluciones aleatorias a generar
  localsearchevals = numero de evaluaciones de BL
  evals=0
  for i in range(0,numsol):
    randsol = solucion aleatoria
    randfitness = fitness(randsol)
```

```
    localsearch(randsol,randfitness,localsearchevals)
    evals += evaluaciones loacalsearch

    if(fitness solución localsearch > bestfitness ):
      bestSolution = solucion localsearch
      bestfitness = fitness localsearch

  return(bestsolution,bestfitness,evals)
```

El algoritmo BMB genera varias soluciones iniciales aleatorias y aplica Búsqueda Local a cada una de ellas. En la implementación se generan 5 soluciones y a cada una se le aplica BL con 2000 evaluaciones y ratio 0.4. Finalmente, se conserva la mejor solución encontrada entre todos los arranques. Esta estrategia reduce la dependencia de la solución inicial al explorar diferentes zonas del espacio de búsqueda.

3.3 Algoritmo ILS

En ILS se parte de una solución inicial mejorada con Búsqueda Local y, a partir de la mejor solución encontrada, se aplica una mutación que perturba parcialmente el vector. Tras la perturbación se vuelve a aplicar Búsqueda Local y se actualiza la mejor solución si se obtiene un fitness superior. En la implementación se realizan 4 iteraciones con 2000 evaluaciones de BL por iteración.

```
# Pseudocódigo del algoritmo ILS
function ils(maxevals):
  localsearchevals = evaluaciones para BL
  niter = numero de iteraciones
  evals = 0

  current = solucion aleatoria
  currentfitness = fitness(current)
  evals += 1

  localSearch(current, currentfitness, localsearchevals)
  evals += evaluaciones BL
  # el resultado se queda igual que el inicial o mejor
  bestsolution = resultado local search
  bestfitness = result.fitness

  for i in range(0, niter):
    #aplico la mutación a la mejor solución
    mutsol = mutation(bestsolution, 0.2)
    localSearch(mutsol, fitness(mutsol), localsearchevals)
    evals += evaluaciones BL
    if fitness BL > bestfitness:
      bestfitness = fitness BL
      bestsolution = solución BL

  return (bestsolution, bestfitness, evals)
```

3.4 Algoritmo Enfriamiento simulado

El algoritmo de Enfriamiento Simulado es una búsqueda local probabilística que permite aceptar empeoramientos con cierta probabilidad para escapar de óptimos locales. Se define un número máximo de vecinos por temperatura, un número máximo de éxitos y una temperatura inicial calculada en función del fitness. En cada iteración se generan vecinos moviendo parte del peso de una empresa a otra, se evalúa el candidato y se acepta si mejora o con probabilidad $\Delta(T)$. La temperatura se actualiza con un esquema de enfriamiento Cauchy hasta que no se obtienen éxitos o se superan las evaluaciones.

```
# Pseudocódigo del algoritmo ES
function es(problem, current, currentfitness, maxevals):
  evals = 0
  problemSize = tamaño del problema
  (low, high) = restricciones del problema

  maxVecinos = 10*problemSize
  maxExitos = problemSize
  nEnfriamientos= maxevals / maxVecinos
  phi = 0.3
  mu = 0.2
  tempFinal = 1e-3
  tempInicial = (mu * abs(currentfitness)) / -log(phi)
  # aseguro que la temperatura inicial no sea demasiado baja
  while tempInicial <= tempFinal:
    currentsolution = solucion aleatoria
    currentfitness = fitness(current)
    evals += 1
    tempInicial = (mu * abs(currentfitness)) / -log(phi)

  beta = (tempInicial - tempFinal) / (nEnfriamientos * tempInicial *
tempFinal)
  temp = tempInicial

  neighbors = generar vecindario de parejas i,j
  bestsolution = currentsolution
  bestfitness = currentfitnessfitness
  hayExito = true

  while evals < maxevals and hayExito:
    vecinosGenerados = 0
    exitos = 0
    mezclar(neighbors)
    pos = 0

    while vecinosGenerados < maxVecinos and exitos < maxExitos and
          evals < maxevals and pos < neighbors.size:
      (i, j) = neighbors[pos]
      # compruebo que esté en el rango
      if current[i] <= low or current[j] >= high:
        pos += 1
        continue
      # selecciono la cantidad que puedo extraer de la posición seleccionada
```

```
      ratio = min(1 - low / current[i], (high - current[j]) / current[i],
0.3)

      #compruebo que no sea 0
      if ratio == 0:
        pos += 1
        continue

      # guardo los valores antiguos y actualizo los vecinos
      vecinosGenerados += 1
      valori = current[i]
      valorj = current[j]
      current[i] = (1 - ratio) * valorI
      current[j] = valorJ + ratio * valorI

      if es válida current:
        candidateFitness = fitness(current)
        evals += 1
        incremento = currentfitness - candidateFitness
        u = aleatorio entre 0 y 1
        ## selecciono la solución si mejora o si empeora la
        ## selecciono con probabilidad u
        if incremento < 0 o u <= exp(-incremento / temp):
          currentfitness = candidateFitness
          exitos += 1
          pos = 0
          if currentfitness > bestfitness:
            bestfitness = currentfitness
            bestsolution = current
        else:
          # si no acepto la solución reestablezco los valores
          current[i] = valori
          current[j] = valorj
          pos += 1
      else:
        current[i] = valori
        current[j] = valorj
        pos += 1
    # actualizo la temperatura
    temp = temp / (1 + beta * temp)
    hayExito = exitos > 0

  return (bestsolution, bestfitness, evals)
```

El código que se usa para generar vecinos es el mismo que la BL y es el siguiente:

```
neighbors vector de parejas
 for i in range(0,nvecinos):
  for i in range(0,nvecinos):
     if (i != j):
       añado la pareja (i,j) al vector
```

3.5 Algoritmo ILS-ES

ILS-ES combina ILS con Enfriamiento Simulado en lugar de Búsqueda Local. La estructura es idéntica a ILS, pero la fase de mejora usa ES durante 2000 evaluaciones. Esto introduce una exploración más amplia que la BL al aceptar movimientos no necesariamente mejores según la temperatura actual.

```
# Pseudocódigo del algoritmo ILS
function ilses(maxevals):
  localsearchevals = evaluaciones para ES
  niter = numero de iteraciones
  evals = 0

  current = solucion aleatoria
  currentfitness = fitness(current)
  evals += 1

  localSearch(current, currentfitness, localsearchevals)
  evals += evaluaciones ES
  # el resultado se queda igual que el inicial o mejor
  bestsolution = resultado ES
  bestfitness = result.fitness

  for i in range(0, niter):
    #aplico la mutación a la mejor solución
    mutsol = mutation(bestsolution, 0.2)
    localSearch(mutsol, fitness(mutsol), localsearchevals)
    evals += evaluaciones ES
    if fitness ES > bestfitness:
      bestfitness = fitness ES
      bestsolution = solución ES

  return (bestsolution, bestfitness, evals)
```

4 Estructura del código

El código de la práctica está organizado en una estructura modular que facilita la compilación y ejecución. A continuación se describe la organización del proyecto y el manual de usuario para compilar y ejecutar el programa.

4.1 Organización de archivos

El proyecto está organizado de la siguiente manera:

- `software/` : Directorio raíz que contiene todo el código fuente
  - `main.cpp` : Archivo principal que ejecuta los algoritmos
  - `src/` : Contiene las implementaciones (.cpp)
    - `ambest.cpp` : Implementación de applyLocalSearch para amBest
    - `pportfolio.cpp` : Implementación de la clase del problema
    - `randomsearch.cpp` : Implementación del algoritmo de búsqueda aleatoria

  - `greedy.cpp` : Implementación del algoritmo Greedy
  - `localsearch.cpp` : Implementación del algoritmo de búsqueda local
  - `es.cpp` : Implementación de el algoritmo ES
  - `ils.cpp` : Implementación de el algoritmo ILS
  - `ilses.cpp` : Implementación de el algoritmo ILS-ES
  - `bmb.cpp` : Implementación de el algoritmo BMB
- `include/` : Contiene los archivos de cabecera (.h y .hpp)
  - `am.h`: Clase base para los algoritmos meméticos
  - `amall.h`, `ambest.h`, `amrand.h`: Clases de los algoritmos meméticos.
  - `pportfolio.h`, `greedy.h`, `localsearch.h`, `randomsearch.h`: Cabeceras de los algoritmos P1.
  - `es.h` : Definición de el algoritmo ES
  - `ils.h` : Definición de el algoritmo ILS
  - `ilses.h` : Definición de el algoritmo ILS-ES
  - `bmb.h` : Definición de el algoritmo BMB
  - `problem.h` : Clase base abstracta
  - `mh.h` : Clase base para los algoritmos de metaheurísticas
  - `solution.h` : Estructura para representar soluciones
  - `random.hpp`, `utils.hpp` : Utilidades auxiliares
  - `Eigen/` : Biblioteca para operaciones con matrices y vectores
- `data/` : Contiene los archivos CSV con los datos de entrada
- `CMakeLists.txt` : Archivo de configuración de CMake
- `script.sh` : Script de compilación automatizada
- `execute_all.sh` : Script para ejecutar el programa en los 3 dataset a la vez.
- `Makefile` : Archivo generado automáticamente por CMake

4.2 Compilación

El proyecto utiliza **CMake** como sistema de compilación automática. Se proporcionan dos modos de compilación:

Opción 1: Usando el script de compilación

El script **script.sh** automatiza el proceso de compilación:

```
# Para compilar en modo debug (con símbolos de depuración):
./script.sh debug

# Para compilar en modo release (optimización máxima):
./script.sh release
```

- **Modo debug**: Compila con la flag -O0 (sin optimización) y -g (símbolos de depuración), para detectar errores y depurar.
- **Modo release**: Compila con -O3 (máxima optimización) y -DNDEBUG (desactiva comprobaciones de Eigen), con esta compilación se han hecho las pruebas.

Opción 2: Usando CMake directamente

```
# Configurar el proyecto en modo release:
cmake CMakeLists.txt -DCMAKE_BUILD_TYPE=Release
```

```
# Compilar:
make
```

Tras cualquiera de estos procesos se generará el ejecutable **program** en el directorio software/.

4.3 Manual de usuario: Ejecución

Una vez compilado el programa, se puede ejecutar de las siguientes formas:

Ejecución estándar (comparación de algoritmos):

```
./program <ruta_archivo_csv> <low> <high> <evaluaciones> <seed> <output_csv>
```

Parámetros:

- **ruta_archivo_csv**: Ruta al archivo CSV con los datos de retornos diarios (ejemplo: data/portfolio.csv)
- **low**: Porcentaje mínimo de inversión permitido por empresa (ejemplo: 0.005)
- **high**: Porcentaje máximo de inversión permitido por empresa (ejemplo: 0.08)
- **evaluaciones**: Número de ejecuciones de cada algoritmo con diferentes seeds
- **seed**: Semilla inicial para el generador de números aleatorios (se incrementa en 10 por cada evaluación)
- **output_csv** : Archivo CSV donde se guardan los fitness de todas las ejecuciones

El parámetro output_csv es opcional, si no se pone entonces no se guardan los datos en un fichero y solo aparecen las medias.

Ejemplo de ejecución:

```
./program data/syp_100.csv 0.005 0.05 50 42 syp100.csv
```

Este comando ejecutará cada uno de los tres algoritmos (Random Search, Greedy y Local Search) 5 veces con seeds diferentes (42, 52, 62, 72, 82). Al finalizar mostrará para cada algoritmo:

- Fitness medio obtenido
- Fitness del año 2025
- Beneficio obtenido en el año 2025
- Número medio de evaluaciones realizadas
- Tiempo medio de ejecución (en segundos)

Al final creará un .csv con los datos de la ejecución

Ejecución simplificada (valores por defecto):

```
./program data/syp_100.csv 0.005 0.05
```

Ejecuta una sola evaluación con seed 42.

Ejecución de ayuda:

```
./program
```

Muestra el mensaje de ayuda con las instrucciones de uso.

5 Experimentos y análisis de los resultados

5.1 Casos empleados y valores de los parámetros.

Para comparar los resultados de los algoritmos se han usado los beneficios diarios de las empresas del S&P500, S&P100 e IBEX35 con los siguientes parámetros.

- $\boldsymbol{\lambda}$= 500 (usado en la función fitness)
- **seed** = 42 (por cada ejecución del algoritmo se incrementa 10 unidades)
- **low** = 0.005 (para todos los datos)
- **high**(para IB35) = 0.08
- **high**(para S&P100) = 0.05
- **high**(para S&P500) = 0.02
- **totalevaluations** = 50

La ejecución se ha hecho como se indicó en el apartado anterior, cambiando en los argumentos del programa el archivo de entrada de datos y ajustando su respectivo valor **high** correspondiente a cada archivo.

5.2 Resultados obtenidos

Algoritmo	**2015-2024 Fitness**	**2025 Fitness**	**Beneficio**	**Evaluaciones**	**Tiempo (segundos)**
Greedy	−4.558	−3.254	0.040	1	0.001
Random	−4.980	−3.378	0.052	10000	0.005
BL	−4.608	−3.115	0.020	1517.36	0.001
AM-BEST	−4.494	−3.084	0.032	10000	0.005
BMB	−4.566	−3.104	0.022	6990.3	0.001
ILS	−4.598	−3.113	0.020	5596	0.004
ES	−4.594	−3.113	0.022	10000	0.003
ILS-ES	−4.592	−3.112	0.022	10000	0.004

Tabla 1: Resultados de Ibex 35 (Medias)

Algoritmo	2015-2024 Fitness	2025 Fitness	Beneficio	Evaluaciones	Tiempo (segundos)
Greedy	−2.835	−3.691	0.092	1	0.001
Random	−3.408	−4.174	0.088	10000	0.02
BL	−3.118	−3.888	0.096	8984.76	0.01
AM-BEST	−2.814	−3.755	0.101	10000	0.044
BMB	−3.023	−3.837	0.092	10000	0.022
ILS	−3.097	−3.877	0.093	10000	0.014
ES	−3.103	−3.895	0.095	10000	0.048
ILS-ES	−3.085	−3.885	0.092	10000	0.063

Tabla 2: Resultados de S&P100 (Medias)

Algoritmo	2015-2024 Fitness	2025 Fitness	Beneficio	Evaluaciones	Tiempo (segundos)
Greedy	−2.749	−3.776	0.013	1	0.001
Random	−3.638	−4.389	0.026	10000	0.253
BL	−3.517	−4.324	0.010	10000	0.220
AM-BEST	−3.220	−4.156	0.005	10000	0.456
BMB	−3.663	−4.450	0.013	10000	0.238
ILS	−3.655	−4.404	−0.001	10000	1.345
ES	−3.451	−4.343	0.012	10000	2.606
ILS-ES	−3.541	−4.330	0.015	10000	0.170

Tabla 3: Resultados de S&P500 (Medias)

Los valores presentados en la tabla corresponden a las medias de 50 ejecuciones independientes con diferentes semillas aleatorias.

5.3 Boxplots de los resultados

A continuación se muestran los boxplots de los resultados obtenidos para cada conjunto de datos tras 50 ejecuciones de cada algoritmo:

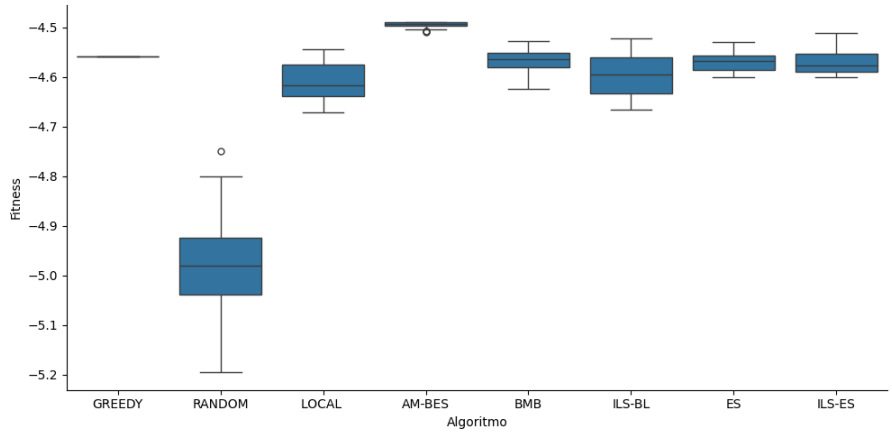


Figura 1: Boxplot de resultados para Ibex 35

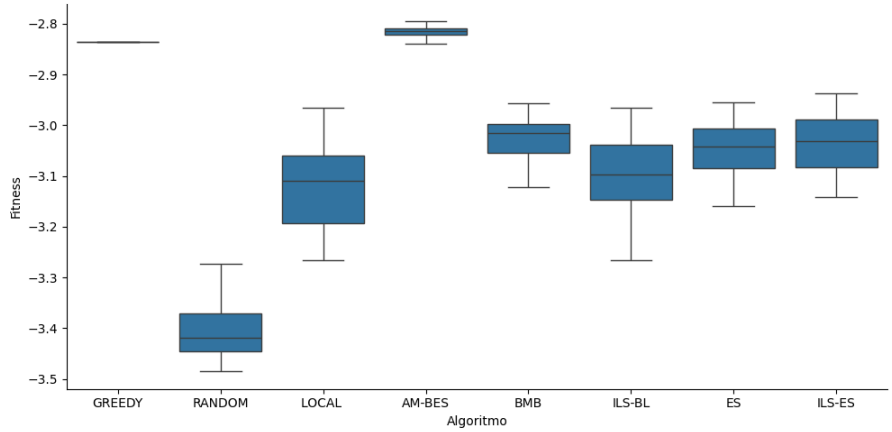


Figura 2: Boxplot de resultados para S&P100

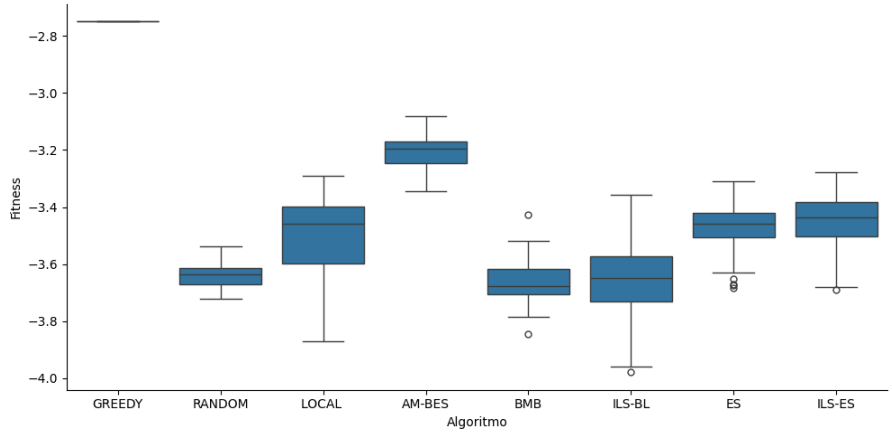


Figura 3: Boxplot de resultados para S&P500

5.4 Gráficas de evolución del fitness durante las ejecuciones

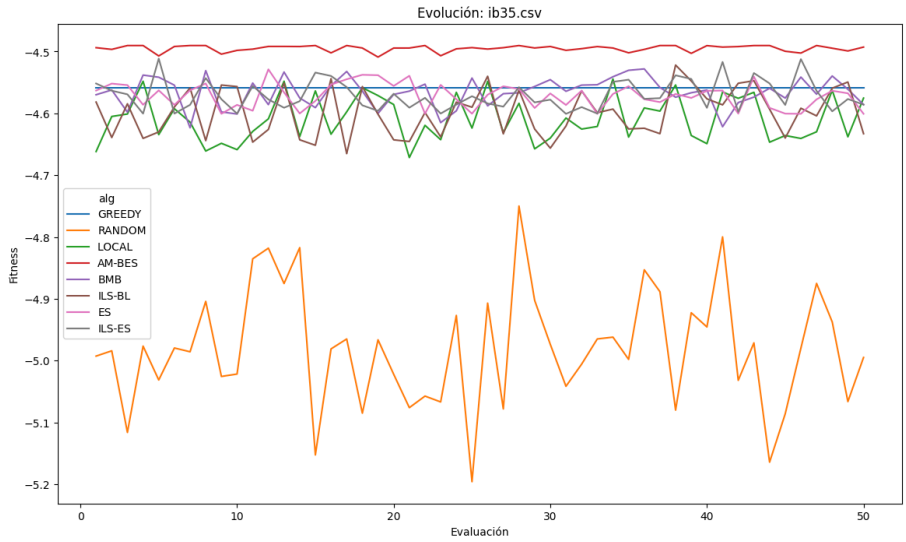


Figura 4: Comparación fitness para Ibex 35

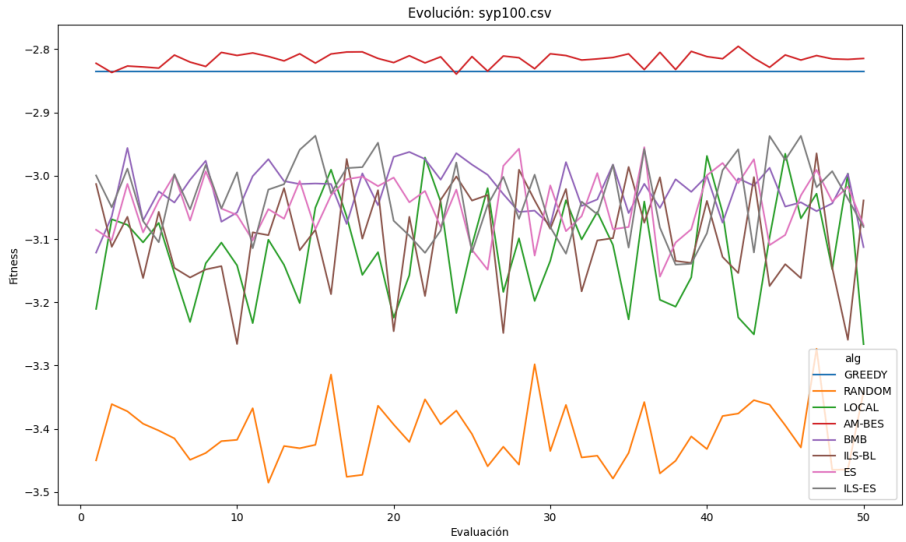


Figura 5: Comparación fitness para S&P100

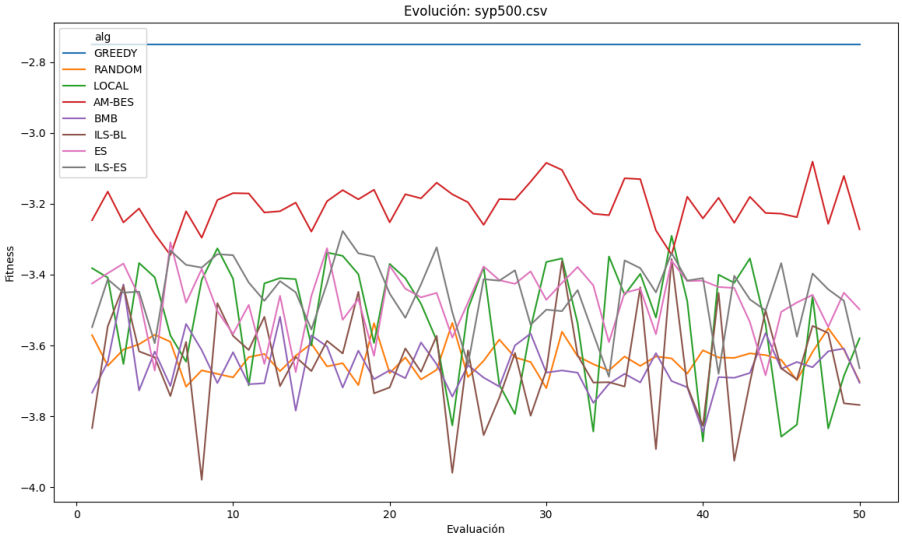


Figura 6: Boxplot de resultados para S&P500

6 Análisis de los resultados

Al observar los resultados obtenidos, podemos ver que el comportamiento de los algoritmos cambia bastante según el tamaño del dataset.

6.1 Comparación global de fitness (2015-2024) y gráficas

6.1.1 Enfriamiento simulado

Podemos ver que el enfriamiento simulado supera a todos los algoritmos de la práctica 3 en cuanto a fitness en el dataset del S&P500, pero no logra superar al mejor algoritmo de la práctica 2 (AM-BEST). Esto se debe a que el AM-BEST es capaz de explotar y explorar el espacio de soluciones mucho mejor que el ES.

Si lo comparamos con la BL, vemos que aunque el enfriamiento simulado es mejor, la diferencia en el fitness no es tan grande para que esta mejora suponga una gran diferencia. Esto se debe a que, para que el enfriamiento simulado gane a la búsqueda local, debemos tener datasets donde empeorar la solución actual nos pueda llevar a una solución mejor, si el espacio no tiene mínimos locales que sean profundos separando los máximos, el permitir empeorar una solución como se hace en el ES no sacará tanta ventaja frente a la búsqueda local, que no permite empeorar las soluciones. Vemos que en el IB35 y en el S&P100 la diferencia entre BL y ES no es tan grande, mientras que en S&P500 la diferencia en fitness sí es más grande. Esto se debe a que en el dataset de S&P500 permitir empeorar soluciones sí da más ventaja. También debemos recordar que al ser un dataset más grande, la solución inicial influye más, esto lo podemos comprobar en los boxplots, donde vemos que cuanto más grande es el dataset, más grande es el rango generado por el boxplot. Si observamos los boxplots de BL y ES, vemos que el ES tiene menos variabilidad; por tanto, le afecta menos la decisión de la solución inicial que a la BL. Si nos fijamos en los gráficos de líneas (rojo y verde), vemos que el ES suele tener un fitness más estable a lo largo de las ejecuciones que la BL.

Si comparamos el ES con los demás algoritmos de la práctica 3 vemos que BMB le gana en el IB35 y S&P100 ya que al ser espacios más pequeños a el BMB le es suficiente con las iteraciones asignadas a cada solución para superar al ES.Podemos observar también que a nivel de tiempo de ejecución ES es mucho más costoso que los demás.

6.1.2 ILS vs BMB vs ILS-ES

En este apartado compararé los algoritmos ILS, ILS-ES y BMB, que tienen estructuras parecidas y sus resultados son similares. Mirando los boxplots y las gráficas de líneas podemos ver que en estos algoritmos influye bastante la selección de las soluciones iniciales, viendo que cuanto mayor es el dataset, más repercusión tienen. En los datasets de IB35 y S&P100 los resultados son bastante buenos, ya que BMB llega a superar a la búsqueda local y el ILS y ILS-ES consiguen superarala pero no con tanta ventaja. En el dataset de S&P500, si nos fijamos en los boxplots, presentan resultados con una variabilidad mucho mayor que la de random por parte de los tres algoritmos, en el BMB esto se debe a que, al escoger las soluciones, como debemos dividir las iteraciones de la BL en 5 soluciones, la BL tiene menos oportunidad de encontrar una solución buena con

ese rango de iteraciones ya que el espacio es muy grande, por tanto dependerá mucho de las soluciones seleccionadas. Por parte del ILS, vemos que el que usa ES tiene una caja más pequeña en el S&P500; esto se debe a que ES funciona mejor en este dataset; por el contrario, cuando aplicamos la BL, sí se ve mucha más variabilidad.

Si nos fijamos en la tabla de resultados, vemos que en los datasets de S&P100 e IB35 BMB consigue superar a ES. Por otro lado, ILS no consigue superar claramente nunca a ES ni a BMB en ninguna de sus versiones ya que en los datasets medianos la diferencia que obtiene es mínima.

6.1.3 Comparación general

Como resultado de los apartados anteriores, podemos ver que en el periodo 2015-2024 los algoritmos de esta práctica que mejores resultados obtienen son el ES y el BMB. Por el contrario, ILS no consigue destacar sobre ninguno de los anteriores, esto puede deberse a la mutación usada, ya que al desordenar un 20% del vector podemos estar poniendo pesos altos en empresas muy malas y esto perjudica bastante a nuestra solución. Dentro de las dos variantes de ILS, vemos que la que mejor funciona es la ILS-ES ya que obtiene mejores resultados de media y presenta una menor dependencia de las soluciones iniciales escogidas. Ninguno de estos algoritmos consigue superar al mejor de la práctica 2 ni al Greedy, esto se debe a que la heurística del Greedy es bastante buena para este problema y el algoritmo de la práctica 2 es mucho más potente que los algoritmos que estamos usando en esta práctica y con las restricciones de evaluaciones que tenemos.

6.2 Resultados en 2025 y beneficio

Al evaluar en 2025 se ve que, dentro de BL, ES, BMB, ILS e ILS-ES, el algoritmo con mejor fitness no siempre coincide con el que mayor beneficio obtiene. En IBEX35 las diferencias son pequeñas y los cinco se comportan de forma muy parecida, lo que nos indica un mercado más estable y menor ventaja de los mecanismos de exploración. En S&P100 no se relaciona directamente mayor fitness con mayor beneficio, lo que indica que las soluciones que obtienen algoritmos como el BMB o el random a pesar de no tener el mejor fitness obtienen la mejor solución , esto se debe a que son soluciones más arriesgadas que obtienen mejores resultados pero que tienen peor fitness . En S&P500 el rendimiento empeora para todos y la variabilidad de los algoritmos pesa más . Esto confirma que la generalización a 2025 es especialmente complicada en el dataset más grande.Empeorando sobre todo los resultados de algoritmos como ILS y ES.

6.3 Conclusiones

Tras analizar los distintos algoritmos vemos que los ganadores en cada dataset son : BMB en IBEX35 y S&P100 y ES en S&P500.Los resultados son bastante ajustados entre BMB y ES , vemos que la opción de búsqueda multiarranque es más eficiente en espacios pequeños ya que con las evaluaciones que destina a la búsqueda local le es suficiente para encontrar una solución buena , mientras que a medida que se agranda el espacio , el BMB pierde fuerza y el algoritmo de Enfriamiento simulado obtiene los mejores resultados sobre los demás. También podemos concluir que en este caso ILS funciona mejor cuando se aplica enfriamiento simulado en cada ocasión en vez de búsqueda local.

Por tanto como conclusión sacamos que los algoritmos analizados en la práctica 3 no consiguen superar al mejor de la práctica 2, y en general , quedan bastante peor que este. Por párte de la práctica 1 , en datasets medianos , superan al random y consiguen igualar a la búsqueda local la mayoría de veces (como observamos en los gráficos de líneas), pero una vez agrandamos el dataset , el comportamiento es más impredecible y no suelen quedar por encima de del Random.